

Test instruction

Welcome to the resit exam of the course Multi-Agent Systems (XB_0024). This exam will test your knowledge and understanding of the material provided to you and presented in the lectures, the book of Blackburn, Bos and Striegnitz *Learn Prolog Now!* (Chapters 1 to 6, 10 and 11), and the MARBEL Guide. It is not allowed to use materials such as books, scientific papers or slides during the exam. You will have 2:45 hours to complete the exam.

The following tools are permitted:

- Pen and drafting paper

For questions about the content of the test you can contact the teacher. Information about the review of the exam is provided via Canvas.

If you have **not** signed up for this exam, you will not receive a result. Through VUnet you can object to the fact that you can no longer sign up after the expiry of the registration deadline (and the fact that you will not receive a result for this exam). Submit your appeal online within one week after the exam. More information can be found at www.vu.nl/intekenen.

Good luck!

Welkom bij het hertentamen van het vak Multi-Agent Systems (XB_0024). Deze toets test je kennis en begrip van de stof als gepresenteerd in de colleges, het boek van Blackburn, Bos en Striegnitz *Learn Prolog Now!* (Hoofdstukken 1 tot 6, 10 en 11), en de MARBEL Guide. Het is niet toegestaan om hulpmiddelen zoals boeken, publicaties en slides te gebruiken tijdens de toets. Je hebt 2:45 uur om de toets te maken.

Bij deze toets mag je gebruik maken van de volgende hulpmiddelen:

- Pen en kladpapier

Voor vragen over de inhoud van de toets kun je bij de docent terecht. Informatie over de toetsinzage wordt via Canvas verstrekt.

Als je **niet** hebt ingetekend voor dit tentamen, ontvang je geen resultaat. Via VUnet kun je bezwaar maken tegen het feit dat je niet meer kunt intekenen na het verstrijken van de intekendeadline (en je dus geen resultaat krijgt voor dit tentamen). Dien je bezwaarschrift online in binnen één week na het tentamen. Meer informatie vind je op www.vu.nl/intekenen.

Veel succes!

Prolog queries

Question order: Random

The following questions each give a prolog query, after which you are asked to fill in the outcome that Prolog will give based on the query.

Example:

?- X = a.

X = a.

Question 1 – 22-23_prolog_query3 – 359078.1.0

`crusty(X) = crusty(crab).`

X = crab.

X=crab.

X= crab.

X =crab.

X=crab .

X = crab .

Question 2 – 22-23_prolog_query4 – 359079.1.0

$X+2$ is 5.

false.

false .

False.

False .

false

False

ERROR

Error

ERROR.

Error.

ERROR .

Error .

Question 3 – 22-23_prolog_query5 – 359080.1.0

$a = b$.

false.

False.

ERROR.

Error.

false .

False .

ERROR .

Error .

false

FALSE

ERROR

Error

Question 4 – 22-23_prolog_query6 – 359286.1.0

A = 2+3.

A = 2+3.

A = 2+3

A=2+3.

A=2+3

A =2+3.

A =2+3

A= 2+3.

A= 2+3

A = 2 + 3.

A = 2 + 3

Question 5 – 22-23_prolog_query7 – 359287.1.0

A is 5+5.

A = 10.

A = 10

A=10.

A=10

A= 10.

A= 10

A =10.

A =10

Prolog search

Question order: Fixed

For the next questions, assume the Prolog knowledge base displayed below:

```
f(a,b) .  
f(b,c) .  
f(c,d) .  
r(X,Y) :- f(X,Z), r(Z,Y) .  
r(X,Y) :- f(X,Y) .
```

Give Prolog's (first) answer for each of the following four queries to this knowledge base. The fifth question in this block is of a different type.

Tip: you can maintain visibility of this block text for each of the questions, by clicking 'Show block intro' above a question.

Question 6 – 22-23_prolog_search1 – 359288.1.1

?- r(a,c) .

true.

True.

TRUE.

true

True

TRUE

Question 7 – 22-23_prolog_search2 – 359292.1.1

?- r(A, B) .

A = a, B = d.

A=a, B=d.

A = a, B = d

A=a, B=d

A = a; B = d.

A = a; B = d

A=a; B=d.

A=a; B=d

Question 8 – 22-23_prolog_search3 – 359293.1.1

?- r(c,d) .

true.

True.

TRUE.

yes.

Yes.

true

True

TRUE

yes

Yes

Question 9 – 22-23_prolog_search4 – 359294.1.1

?- r(c,b) .

False

False.

false.

false

FALSE.

FALSE

ERROR.

Error

ERROR

Error.

error.

error

Question 10 – 22-23_prolog_search5 – 359299.1.1

What fact can you add to the given database that would make *any* of the following queries end up in the infinite search branch?

?- r(a,b) .

?- r(X,Y) .

?- r(d,c) .

f(d,a).

f(d,a)

f(d, a).

f(d, a)

Prolog concepts

Question order: Random

The following questions address your knowledge of prolog concepts. Answers are either given in multiple choice or in an open text field.

Question 11 – 22-23_prolog_concepts1 – 359593.2.0

Consider the following predicate and query:

```
addFiveAndHalve(X,Y):- Y is (X+5)/2.  
?- addFiveAndHalve(X,3).
```

Give the outcome to the query, and explain why this is the case.

Grading instruction

Correct outcome (Number of points: 1)

The query will give an 'ERROR: is/2.

Correct explanation (Number of points: 2)

This predicate only leads to a successful outcome when X has been properly instantiated with a number, which is not the case here. (Something along the lines of instantiation should be mentioned).

Question 12 – Prolog_concepts2 – 211812.2.0

Which of the following prolog predicates is defined in a tail-recursive way?

- A** `xyz([],0).`
 `xyz([_|L],NL) :- xyz(L,L2), NL is L2 + 1.`
- B** `xyz([],X,X).`
 `xyz([_|L],OX,L) :- NX is OX + 1, xyz(L,NX,L).`
- C** `xyz([],X,X).` `xyz(X,[],X).`

Question 13 – Prolog_concepts3 – 211817.3.0

Explain how Prolog's negation is different from the classical negation used in first-order logic.

Grading instruction

naff (Number of points: 2)

Mentioning negation as finite-failure / prolog search.

Negation in FOL (Number of points: 1)

Indicating that negation in FOL applies when the negated part is not true or non-existent in the universe.

Prolog programming

Question order: Random

In the following questions you are asked to write a prolog predicate with a certain purpose, or select the proper predicate from a list of options.

For questions where you are asked to write a prolog predicate, you are not allowed to use any built-in predicates. Remember to think of one or more base clauses and one or more recursive clauses. Incorrect predicates might still be rewarded some of the points, if, for example, the base clause is correct.

Question 14 – 21_22_prolog_programming_matchcount – 313926.1.0

Define a prolog predicate `matchCount(L,M,C)`, where C matches the number of terms in list L that are also in list M, with the following query outcomes as result:

```
?- matchCount([a,b,c,d,e,f,g],[a,d,g],C).  
C = 3.
```

```
?- matchCount([a,b,c,d,e,f,g],[h,i],C).  
C = 0.
```

```
?- matchCount([g],[a,b,c,d,e,f,g],C).  
C = 1.
```

You can make use of the built-in prolog predicate `member(Term?,List?)`, which is true if the term Term is a member of List.

Grading instruction

baseclause (Number of points: 1)

% depending on the use of an accumulator, the base clause can be either of the following two:
`matchCount([], M, 0).`

or

```
mCAcc([],M,A,A).
```

Recursive clause 1 (Number of points: 2)

% One recursive clause is dedicated to the scenario where the head of the list is member of M. Depending on the use of an accumulator, this clause can be either of the following two:
`matchCount([H|T], M, C) :- member(H,M), matchCount(T,M,C2), C is C2 + 1.`

or

```
mCAcc([H|T],M,A,C) :- member(H,M), ANew is A + 1, mCAcc(T,M,ANew,C).
```

Recursive clause 2 (Number of points: 1)

% The second recursive clause needs to account for terms that are not member of M. Depending on the use of an accumulator, this clause can be either of the following two:
`matchCount([H|T], M, C) :- not(member(H,M)), matchCount(T,M,C).`

or

```
mCAcc([H|T],M,A,C) :- not(member(H,M)), mCAcc(T,M,A,C).
```

Question 15 – 22-23_prolog_programming1 – 359480.1.0

Give the implementation for a prolog predicate `nr_alpha(L,C)` , which counts the number of alphabetic characters `C` in a list `L`. To check for an alphabetic character, you may use the `char_type/2` predicate:

```
?- char_type(a,alpha).
true.
?- char_type(1,alpha).
false.
```

Some example queries of `nr_alpha`:

```
?- nr_alpha([1,a,b],C).
C = 2.
?- nr_alpha([2],C).
C = 0.
```

Tip: when implementing the predicate in a left-recursive way, the implementation would require three lines: one base clause and two recursive clauses. You are also allowed to implement the predicate in a tail-recursive way. Do not forget to include a wrapper clause in that case, so as to maintain an arity of two for the predicate.

Grading instruction

base clause (Number of points: 1)

```
nr_alphabetic([],0).
```

recursive clauses (Number of points: 3)

```
% recursive clause in case of alphabetic Head
nr_alphabetic([H|T],C) :- char_type(H,alpha), nr_alphabetic(T,CNew), C is CNew+1.
% recursive clause in case of non-alphabetic Head
nr_alphabetic([H|T],C) :- not(char_type(H,alpha)), nr_alphabetic(T,C).
```

Question 16 – 22-23_prolog_programming2 – 359591.1.1

Which of the rules below is a proper specification of a predicate `dprod(A,B,C)` , that calculates the dot product of list `A` and list `B`: for each index `I`, multiply the terms with this index in list `A` and list `B` and return the sum of those multiplications as `C`.

```
?- dprod([1,2],[4,5],D).
D = 14.
?- dprod([3,5,1],[4,6,2],D).
D = 44.
?- dprod([1,2],[4],D).
false.
```

- A** `dprod([H1],[H2],D) :- D is H1*H2.`
`dprod([H1|T1],[H2|T2],D) :- Dot is H1*H2, dprod(T1,T2,DNew), D is DNew + Dot.`
- B** `dprod([H1],[H2],D) :- D is H1*H2.`
`dprod([H1|T1],[H2|T2],D) :- Dot is H1*H2, D is DNew + Dot, dprod(T1,T2,DNew).`
- C** `dprod([],[],D).`
`dprod([H1|T1],[H2|T2],D) :- Dot is H1*H2, D is DNew + Dot, dprod(T1,T2,DNew).`
- D** `dprod([],[],D).`
`dprod([H1|T1],[H2|T2],D) :- Dot is T1*T2, dprod(H1,H2,DNew), D is DNew + Dot.`
- E** `dprod([],[],D).`
`dprod([H1|T1],[H2|T2],D) :- Dot is T1*T2, D is DNew + Dot, dprod(H1,H2,DNew).`
- F** `dprod([H1],[H2],D) :- D is H1*H2.`
`dprod([H1|T1],[H2|T2],D) :- Dot is H1*H2, dprod(T1,T2,DNew).`

Question 17 – 21_22_prolog_programming_position – 313984.2.0

Define a prolog predicate `position(X,L,R)`, that matches R to the number X inserted in the ordered list L (from high to low) such that the ordering is maintained. Some outputs of queries to this predicate are shown below:

```
?- position(3,[7,4,2,1],R).  
R = [7, 4, 3, 2, 1].
```

```
?- position(3,[2,1],R).  
R = [3, 2, 1].
```

```
?- position(3,[7,4],R).  
R = [7, 4, 3].
```

```
?- position(3,[],R).  
R = [3].
```

In your implementation, define three clauses that account for the following conditions:

- A first base clause where L is empty
- A second base clause where X is equal to or higher than the head of L
- A recursive clause where X is lower than the head of L

Implement these conditions in the clauses, and think of their further specification so that the `position` predicate results in the intended insert of X.

First base clause: `position(X,[],[X]).`

Second base clause: `position(X,[H|T],R) :- X >= H, R = [X,H|T].`

Recursive clause: `position(X,[H|T],R) :- X < H, position(X,T,R2), R = [H|R2].`

[Alphanumeric]

position(X,[],[X]).

position(X,[],R) :- R = [X].

position(X,[],[X])

position(X,[],[X]) .

position(X,[],R) :- R=[X].

position(X,[],R) :- R=[X]

position(X,[],R) :- R = [X]

[Alphanumeric]

position(X,[H|T],R) :- X >= H, R = [X,H|T].

position(X,[H|T],R) :- X >= H, R = [X,H|T]

[Alphanumeric]

position(X,[H|T],R) :- X < H, position(X,T,R2), R = [H|R2].

Agents and MARBEL: Basics

Question order: Fixed

This block features open text questions about knowledge on basic Agents and MARBEL.

Question 18 – 22-23_MARBEL_Basics_Resit1 – 359962.1.0

Explain what it means for an environment to be *deterministic/stochastic* observable.

Grading instruction

Criterion 1 (Number of points: 2)

Effects of actions (1p) on the environment state are (not) completely predictable (1p).

Question 19 – 22-23_MARBEL_Basics_Resit2 – 359963.1.0

Mention two further properties that environments can have which are relevant for designing agents.

Grading instruction

Criterion 1 (Number of points: 2)

Further properties: fully/partially observable, static/dynamic, discrete/continuous, single/multi agents.

Question 20 – 22-23_MARBEL_Basics_Resit3 – 359964.1.0

In a multi-agent system, to split the work between multiple agents each of those agents will need to do something differently. If each of these agents execute the same programs (code), however, it becomes a problem how to divide the work between these agents (and to break ties on which agent does what). What is a simple solution that you can use to assign different tasks to different agents in such a multi-agent system?

Grading instruction

Criterion 1 (Number of points: 4)

The names of agents can be used (3p) together with a simple mechanism such as alphabetic sorting for ordering these names (1p).

Programming in MARBEL

Question order: Fixed

This assignment is about a simple cleaning robot that is deployed in a house where different rooms are represented as different places in the house. The robot is able to deposit garbage in a bin located in the kitchen. The robot can always move freely from one place to another in the house. Carefully analyze the MARBEL agent system in the Figure below:

Remark 1: Important for the MARBEL programming questions: When you add code in the program templates, clearly specify the position in the code where you want to add something.

If you want to *modify* a line, for example, number 11, in a module named 'file.mod2g', and replace it with code '...', use the following notation:

```
file.mod2g 11 ...
```

If you want to place *new* code '...' in some module file between two line numbers, for example, 23 and 24, use the following notation:

```
file.mod2g 23.1 ...
```

```
file.mod2g 23.2 ...
```

```
file.mod2g 23.3 ...
```

```
_____ cleaningRobot.mas2g _____
1 use "sensors-and-actuators.jar".
2
3 define cleaningRobot {
4   use cleaningInit for init.
5   use cleaning for decisions.
6
7   ...
8   ...
9 }
10
11 launchpolicy {
12   when * launch cleaningRobot.
13 }
_____ cleaning.pl _____
1 :- dynamic in/1, battery/1, carrying/0.
2
3 recharge :- ...

_____ cleaningInit.mod2g _____
1 use cleaning.
2
3 module cleaningInit {
4   ...
5 }
_____ cleaning.mod2g _____
1 use cleaning.
2
3 order=linearrandom.
4
5 module cleaning {
6   % If robot carries garbage, then put it in the bin.
7   if carrying, in(Place), hasBin(Place) then putInBin.
8   % If robot carries garbage, then go to place with bin.
9   if carrying, hasBin(Place) then goTo(Place).
10  % If robot sees garbage, collect it.
11  if garbage(Place), ... then collect.
12  % Go somewhere.
13  if place(Place) then goTo(Place).
14 }
```

Follow the instructions given in Remark 1 in the Figure.

Question 21 – 22-23_MARBEL_Programming_Resit1 – 359953.1.0

The house has different rooms (places), including: a kitchen, a living (room), a hall, a bedroom, and a bathroom. Add an action rule to the code in the most appropriate place (indicate in which line) to make sure that the agent knows these places, i.e. instantiated facts of the form `place(Place)` are added to its database.

Grading instruction

Criterion 1 (Number of points: 5)

```
deliveryInit.mod2g 4 if true then insert(place(kitchen), place(living), place(hall), place(bedroom),  
place(bathroom)).
```

It is always fine to use forall-do instead of if-then. If some mistake(s) in rule but more or less still ok, deduct 1pt. 1pt for putting rules in correct place, i.e. the init module. Also OK to use only a single rule, such as the second one above.

Question 22 – 22-23_MARBEL_Programming_Resit2 – 359954.1.0

The kitchen has a bin to put garbage in and in the kitchen there is a charger for recharging the robot. Use the `hasCharger/1` and `hasBin/1` predicates to add an action rule to the code in the most appropriate place to add these two relevant facts to the agent's database.

Grading instruction

Criterion 1 (Number of points: 2)

```
deliveryInit.mod2g 4.1 if true then insert(hasCharger(kitchen), hasBin(kitchen)).
```

Question 23 – 22-23_MARBEL_Programming_Resit3 – 359955.1.0

Assume that `garbage/1` is a *send always* percept informing the agent that when it receives, for example, `garbage(bedroom)` there is garbage to collect in the bedroom. Add a channel for handling this percept with the appropriate handler to the code and indicate where you would add this.

What else needs to be added to the program to make it work without problems (again indicate also where)?

Grading instruction

Criterion 1 (Number of points: 5)

1. In `cleaningRobot.mas2g`, line 7 replace `garbage/1`.

2. The predicate needs to be declared. In `cleaning.pl` line 1 :- `dynamic in/1, battery/1, garbage/1, carrying/0`.

Correct place `mas2g` (2p) and correct handler 'replace' (1p). Indication that `delivery/1` predicate needs to be declared (1p) and this needs to be added to line 1 in `cleaning.pl` (1p). Just the phrase `In cleaning.pl line 1 :- dynamic in/1, battery/1, garbage/1, carrying/0.` is also OK (=2p).

Question 24 – 22-23_MARBEL_Programming_Resit4 – 359956.1.0

The `in/1` and `battery/1` predicates are send on change percepts informing the agent about whether it is in a particular place and the battery level, respectively. Add an appropriate handler for this percept and indicate where this should be put in the code.

Grading instruction

Criterion 1 (Number of points: 3)

```
In cleaningRobot.mas2g line 8 update in/1.  
In cleaningRobot.mas2g line 8.1 update battery/1.
```

See for point distribution previous question part.

Question 25 – 22-23_MARBEL_Programming_Resit5 – 359957.1.0

The recharge level for the robot's battery is set to 10. That is, if the battery level is less than 10 the robot should recharge its battery. The idea is to use the recharge predicate to check the battery level but it has not yet been defined in the `cleaning.pl` Prolog file, see line 3. Provide a clause to define this predicate.

Grading instruction

Criterion 1 (Number of points: 2)

In `cleaning.pl` line 3 `recharge :- battery(Level), Level<10.`

2pts for this rule. Deduct 1pt if a (minor) mistake is made.

Question 26 – 22-23_MARBEL_Programming_Resit6 – 359958.1.0

The `cleaningRobot` agent can only collect (at most one piece of) garbage (at the same time) if it is in a place where there is garbage. Assume the `collect` action is instantaneous and the robot is carrying garbage when it performs this action. In other words, no percept is needed (and also not received) but the agent needs to keep track of whether the agent is carrying garbage itself. Modify the program to make sure that the agent only performs the `collect` action when it is in a place where there is garbage and the query `carrying` succeeds when the agent has just performed the `collect` action.

Grading instruction

Criterion 1 (Number of points: 4)

In `cleaning.mod2g` line 11 `if garbage(Place), in(Place) then collect + insert(carrying).`

1pt for place to modify (line 11, `cleaning.mod2g`), 1pt for indicating that `in(Place)` needs to be added to the query of this rule, 1pt for indicating that `insert(carrying)` needs to be added to the action part of the rule, and 1pt for using the `+` operator.

Question 27 – 22-23_MARBEL_Programming_Resit7 – 359959.1.0

Why do we not have to add `not(in(Place))` to the end of the query of the rule in line 9 in `cleaning.mod2g`? Explain this by reasoning about the other rules that are in this module.

Grading instruction

Criterion 1 (Number of points: 2)

The rule at line 7 already checks for this. Only if that rule is not applicable, the rule at line 9 would be evaluated. Because the query of the rule at line 7 then must have failed, we can assume that `not(in(Place))` will succeed (as both rules at line 7 and 9 also check for `hasBin(Place)`).

Question 28 – 22-23_MARBEL_Programming_Resit8 – 359960.1.0

Right after performing the action `putInBin` (instantaneously) the robot does not carry any garbage any more. Modify the program to make sure that the robot does no longer believe it is carrying garbage after performing the `putInBin` action.

Grading instruction

Criterion 1 (Number of points: 4)

In `cleaning.mod2g` line 7 `if carrying, in(Place), hasBin(Place) then putInBin + delete(carrying).`

See for distribution of points previous question part.

Question 29 – 22-23_MARBEL_Programming_Resit9 – 359961.1.0

Explain why the last action rule in `cleaning.mod2g` that selects the `goTo(Place)` action to go somewhere should be the last rule in the module and why it will make the robot go to different places in the house.

Grading instruction

Criterion 1 (Number of points: 4)

It should be the last rule because it always is applicable (there always is a place to go to in the house; 2pt). It will go to different rooms because of the order option of the module that is set to `linearrandom` (2pt).